



Voraxia Asteroid Voxelisation Pipeline

Static Mesh to Baked Density Field to Mineable Runtime Asteroid

Version:	v0.1
Date:	29 June 2026
Prepared By:	Coding Custard Studios
Project:	Voraxia
Status:	Design and Technical Foundation

Coding Custard Studios | Voraxia Documentation Standard

1. Why This Exists

The bridge between beautiful asteroid assets and editable, mineable world geometry.

Voraxia should not ask players to mine a decorative shell. An asteroid that looks like an asteroid must also behave like one: it must contain material, reveal deposits when excavated, update its collision, and keep its state authoritative in multiplayer.

THE PROMISE

Imported static meshes become the authored visual starting point. The baked voxel asset becomes the gameplay truth.

THE BOUNDARY

The converter is editor-only. Gameplay never waits for a mesh to be voxelised during live play.

THE PAYOFF

Existing Raptor mining, ore deposits, material sections, fragments and inventory all gain a production-ready source shape.

THE NON-GOAL

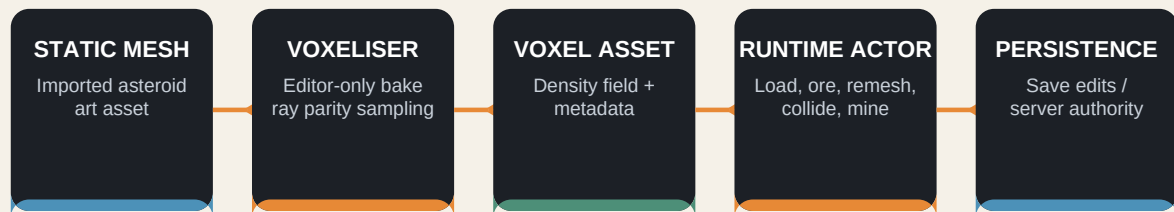
This v0.1 tool does not try to solve planets, large-scale chunk streaming, deformable ship hulls or universal destructibility.

Design stance

Bake slowly, play quickly. The editor can spend time thinking; the live game should only load data, apply edits, remesh the affected region, and replicate the result.

2. End-to-End Pipeline

A deliberately boring conveyor belt. Boring is good when the asteroid is being eaten.



Bake-time steps

- Choose a clean Static Mesh asteroid asset and its desired voxel resolution.
- The voxeliser samples a closed volume, marking points inside or outside the mesh.
- A density field is stored in a reusable data asset with bounds, resolution and source provenance.
- The runtime actor loads the density field, assigns ore metadata, generates mesh sections and supports destructive edits.

SOURCE-OF-TRUTH RULE

The source Static Mesh is art provenance. The baked voxel asset is the runtime gameplay source of truth. Once mined, the runtime asteroid does not return to the original mesh.

3. Baked Asset Contract

The asset should preserve what runtime needs, and nothing that it does not.

FIELD	WHY	v0.1 REPRESENTATION
Source Static Mesh	Traceability and rebuild provenance	Soft object path / source asset reference
Bake Version	Allows migration when the voxeliser improves	Integer or semantic version
Cells Per Axis	Resolution of the density grid	e.g. 32, 48, 64
Voxel Size	World-unit scale of each cell	float, Unreal centimetres
Local Bounds	Original authored mesh bounds / transform fit	FBox or origin + extents
Density Samples	The actual baked scalar field	TArray<float> initially; compressed later
Material Regions	Optional future region IDs from source mesh	Not required for first bake
Default Ore Rules	Optional asset-specific mining profile	Deferred: data-driven pointer/tag

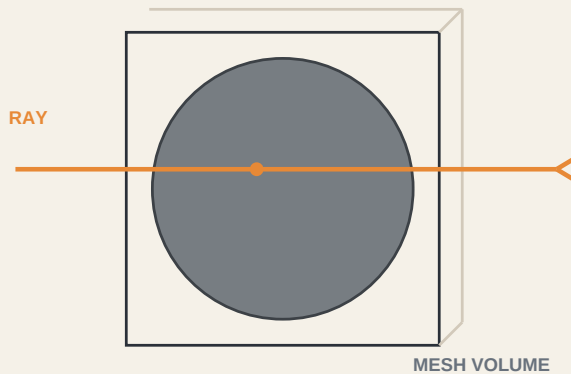
KEEP ORE OUT OF THE BAKE FOR NOW

Density and volume belong to the baked asteroid asset. Ore deposits remain runtime-generated from tags, rules and seeds until authored ore placement is explicitly needed.

4. First Voxelisation Method

Collision-assisted ray parity sampling. Slow in the editor. Transparent to debug.

For v0.1, use a ray-parity test against the source Static Mesh collision. For each sample point, cast a ray in a stable local direction through the mesh. An odd number of intersections means the point is inside; an even number means it is outside.



REQUIRES

Usable mesh collision. For the prototype, enable complex collision as simple on the asteroid asset.

WATCH FOR

Open meshes, accidental holes, non-manifold geometry and paper-thin surfaces. These can cause leaks or ambiguous parity results.

FUTURE UPGRADE

Direct triangle sampling / signed-distance field generation can improve accuracy and bake speed after the baseline pipeline works.

Why this is the right first method

It uses assets you already have in Unreal, is easy to visualise with debug rays, and keeps the first baker independent of raw render-mesh extraction. That makes it ideal for an offline tool whose first mission is reliability.

5. Runtime Integration

The existing mineable sphere proves the systems. The asteroid actor generalises the source shape.

CURRENT PROTOTYPE

AVoraxiaVoxelSphereActor generates a sphere density field, generates ores, builds Marching Cubes geometry, applies Raptor edits, awards yields and cleans small debris.

TARGET ACTOR

AVoraxiaVoxelAsteroidActor loads a baked density field from a Voxel Asteroid Data Asset, then uses the same mining, ore, material-section and cleanup services.

Runtime responsibilities

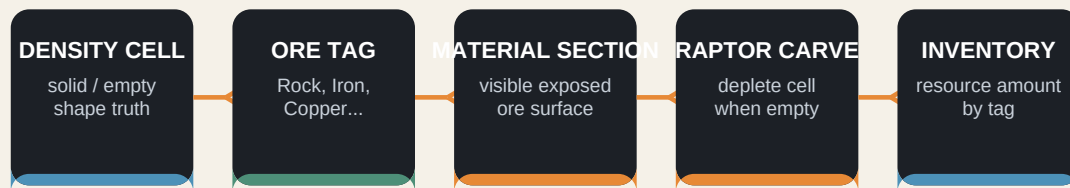
- Load baked density data into mutable per-instance memory. Never edit the source asset at runtime.
- Generate or load ore deposits. Ore tags drive material sections, extraction yields and inventory flow.
- Remesh after a destructive edit. v0.1 can rebuild the whole asteroid; chunk-local remeshing is a later performance milestone.
- Rebuild collision with the mesh. Preserve current no-physics behaviour unless a future debris system opts in.
- Record authoritative state changes server-side for replication and persistence.

DO NOT TURN THE BAKER INTO THE RUNTIME ACTOR

The converter belongs in editor/developer code. The actor belongs in the game module. Keeping them separate protects shipping builds, performance and mental sanity.

6. Ore, Materials and Mining

The voxelised shape enters the same resource loop already proven on the sphere.



Key implementation rule

Ore is not a decorative mesh spawned onto an asteroid. An exposed surface triangle is assigned to the material section belonging to the underlying voxel cell. When the Raptor removes that cell, geometry and material disappear together. The player receives the yield through the same resource tag.

VISUAL TRUTH

Surface-biased deposits appear immediately.
Buried deposits become visible only when excavation reaches them.

GAMEPLAY TRUTH

Rock must remain a trackable resource too.
Every material-bearing cell can contribute a tag and amount to inventory.

7. Milestones and Acceptance Tests

Build the machine in small, inspectable stages.

M0 - Asset validation

Confirm the chosen static meshes have usable collision, reasonable scale, closed volumes and no unexpected holes.

M1 - Data asset

Create UVoraxiaVoxelAsteroidDataAsset containing baked density samples, voxel size, dimensions and source reference.

M2 - Editor baker

Select a Static Mesh, choose resolution, bake a density field with debug sampling, save the asset.

M3 - Runtime load

Create a test asteroid actor that loads baked data and reproduces the source silhouette through Marching Cubes.

M4 - Mining parity

Run the existing Raptor, ore, material sections, collision and inventory flow on a baked asteroid.

M5 - Hardening

Add error reporting, bake metadata, regression assets, performance measurements and server-authority hooks.

v0.1 is complete when

A selected static asteroid mesh can be baked into an asset, spawned as a runtime voxel asteroid, mined with the Raptor, reveal ore materials, update the player inventory, and retain a clear path to server-authoritative persistence.

8. Risk Register and Design Decisions

Known dragons, named before they grow extra heads.

Open or unreliable collision

Ray parity tests may leak. Mitigation: validate source mesh collision before bake and provide a debug volume preview.

Resolution vs fidelity

Higher resolution captures silhouette but grows memory and bake time. Mitigation: expose practical presets and store bake statistics.

Whole-asteroid remesh cost

The existing prototype remeshes all geometry after each carve. Mitigation: keep test asteroids modest, then move to chunks after the first content loop is stable.

Material seam ambiguity

A surface triangle can border multiple material cells. Mitigation: choose a deterministic dominant-cell rule for v0.1 and document it.

Multiplayer divergence

Client-side edits would desynchronise. Mitigation: server owns density edits and yields; clients receive replicated state or remesh deltas later.

9. Immediate Next Actions

The first asteroid is waiting on the loading dock.

1. CHECK COLLISION

Open one of the new asteroid Static Mesh assets. Confirm it has usable complex collision for the prototype. Do not begin the baker until this is known.

2. CHOOSE A TEST MESH

Pick one asteroid with a clear silhouette, modest mesh complexity and no obvious openings. One reliable specimen beats a museum full of weird rocks.

3. BUILD THE DATA ASSET

Add the Voxel Asteroid Data Asset class first. It is the contract between the editor baker and runtime asteroid actor.

4. BUILD THE BAKER

Implement collision-based sampling and save a baked asset. Start with debug output before runtime loading.

Anchor principle

The goal is not to invent procedural asteroids. The goal is to let authored asteroid art be reborn as editable, persistent gameplay matter. That is Voraxia in miniature.